

Nifty 50 Behaviour Analysis Full Project Report

What the system does, how it is built, and the logic behind every part — the data sources, the 50-stock universe, the buy-before / sell-after methodology, the win-rate mathematics, the data-quality work, the dashboard, and the honest limitations.

Universe: the current Nifty 50 (verified 1:1 with NSE)

Data: Yahoo Finance (prices) • NSE India (events) • Screener.in (fundamentals)

Status: local build — for study and education, not investment advice

-
- | | |
|---|--------------------------------|
| 01 Executive summary | 08 How we predict all 4 events |
| 02 What the project is | 09 Data-quality work |
| 03 Data sources & pipeline | 10 The dashboard & tabs |
| 04 The Nifty 50 universe | 11 Performance & caching |
| 05 The dataset today | 12 Scripts reference |
| 06 Core method: buy-before / sell-after | 13 Honest limitations |
| 07 The win-rate mathematics | |

01 Executive summary

This project studies **how Nifty 50 stocks move around their corporate events** — quarterly results, board meetings, corporate actions (dividends/bonuses/splits) and material announcements — to test a simple, disciplined idea: **buy a few days before a known, scheduled event and sell a few days after**. For every stock and event type it brute-force tests **64 timing rules** against the full event history, reports the best rule and its **win rate**, and serves it all through a web dashboard.

50

stocks (exact Nifty 50)

68,589

unique event-dates

106,034

feed rows analysed

50/50

fundamentals + prices

The honest headline finding: past win-rates around events are real but **in-sample and modest** — a description of history, not a proven money machine. The value of the project is the **disciplined, transparent framework** and the clean, verified Nifty 50 dataset behind it.

02 What the project is — the core idea

Markets partly **anticipate** scheduled events (price drifts up into a known results date) and partly **digest** them afterwards (price keeps drifting for days after the news). The strategy tries to sit inside both windows: enter before the event to catch the anticipation, hold through it, and exit after to catch the drift. We never guess *whether* an event will happen — the date is read off the regulatory calendar; we only study the *price behaviour* around that fixed date.

BUY BEFORE

Catch the anticipation

The event date is public in advance, so price tends to drift up into it. Entering **N days before** sits inside that run-up.

SELL AFTER

Catch the drift

The reaction takes several days to play out. Exiting **M days after** harvests the post-event drift instead of selling into the first spike.

03 Data sources & the pipeline

Three external sources feed the system; everything is cached to disk so the analysis runs offline.

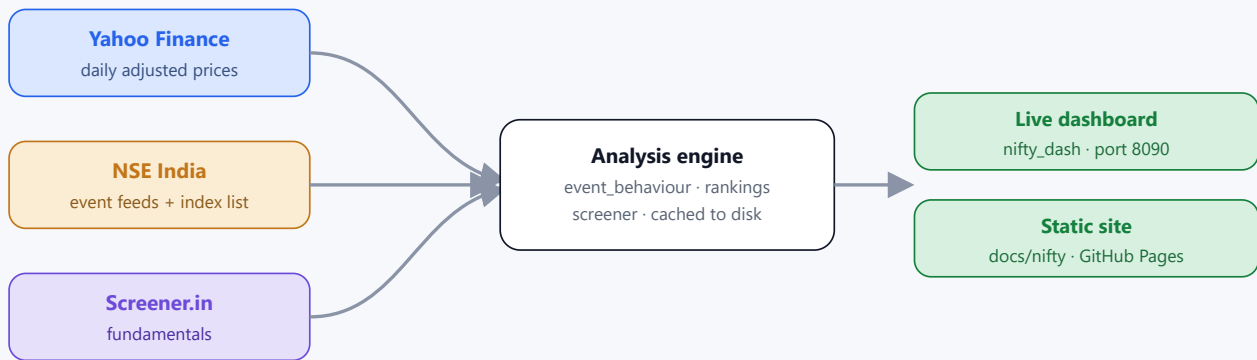


Figure 1. Three sources → cached files → one analysis engine → live dashboard + static snapshot.

DATA	SOURCE	STORED AS	LOGIC / WHY
Daily prices	Yahoo Finance	processed/price_cache/	Adjusted close — corrects for splits/bonuses so the price series is continuous.
NIFTY index	Yahoo (^NSEI)	price_cache/_NSEI	For market-adjusting returns vs the index.
Event feeds	NSE India	<SYM>/*.csv	The calendar of results / board meetings / corporate actions / announcements.
Fundamentals	Screener.in	pn1/ balance_sheet/ ...	P&L, balance sheet, cash flow, ratios, quarterly — for the Screener/Compare tabs.
Index membership	NSE (Company Directory)	download_feeds.py	The authoritative list of the 50 constituents.

04 The Nifty 50 universe — synced & verified

The stock list was re-synced to the **exact current Nifty 50** from the official NSE Company Directory, then verified 1:1 against the NSE MarketWatch export. The previous list had drifted to 52 symbols (stale members never removed through index reshuffles).

+4 added

ETERNAL (ex-Zomato) · INDIGO · MAXHEALTH · TMPV (Tata Motors PV)

New constituents that were missing from the old list — now downloaded and analysed.

-6 removed

BPCL · BRITANNIA · HEROMOTOCO · INDUSINDBK · LTIM · TATAMOTORS

Ex-members dropped from the index. Their data stays on disk — just no longer listed.

Verification result: dashboard universe == NSE Company Directory == NSE MarketWatch export — **all three identical, zero discrepancies**. Every company shown is a current Nifty 50 constituent.

05 The dataset today

After refreshing from the new NSE export and merging the full historical archive, the 50 stocks carry:

FEED	UNIQUE EVENT-DATES	DATE COLUMN	WHAT IT IS
Announcements	63,021	sort_date	Press releases, acquisitions, ratings, outcomes — material disclosures.
Financial results	3,158	broadCastDate	Quarterly/annual results, back to ~2007.
Board meetings	1,553	bm_date	Board-meeting dates (backfilled from announcements to 2006).
Corporate actions	857	exDate	Dividend / bonus / split ex-dates.
Total	68,589		across 106,034 raw feed rows; fundamentals & prices for 50/50.

06 Core method — the buy-before / sell-after trade

For a single event pinned to trading day **T**, the trade is two legs: **buy N trading days before** at the close, hold through the event, **sell M trading days after** at the close. Every event of a type is tested against an **8×8 = 64-rule grid** (N = 1...8, M = 1...8).

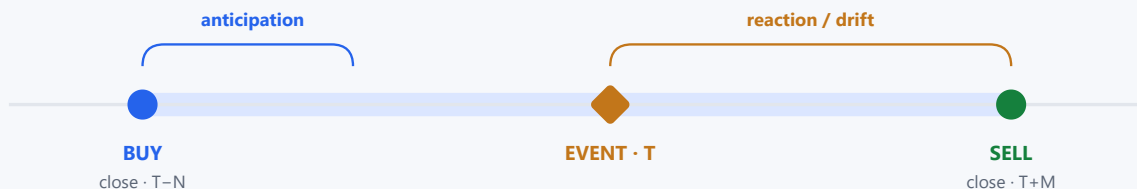


Figure 2. One buy, one sell, held through the event. Days are **trading** days (weekends/holidays skipped).

```
entry = close[T - N] // N days before
exit  = close[T + M] // M days after
return% = ( exit / entry - 1 ) × 100 → WIN if > 0 else LOSS
```

07 The win-rate mathematics

Pool all past events of a type; for each of the 64 (N, M) cells, the win rate is the share that finished green:

```
win_rate(N,M) = ( #events with return% > 0 ) ÷ ( total events ) × 100
```

Choosing the one displayed rule

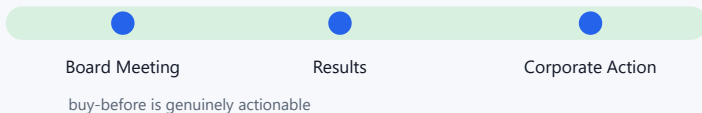
- 1 **Require enough events** — keep cells backed by ~5+ events, so the winner isn't one lucky quarter.
- 2 **Keep the profitable cells** — average return must be positive (else fall back to all).
- 3 **Rank by win rate, then average return** — the top cell's N, M, win% and average become the displayed rule.

This same engine (event_behaviour → build_behaviour → build_rankings) powers the dashboard, the rankings and the static site, so every number is consistent.

08 How we “predict” all four events

We never forecast *whether* an event will happen. For scheduled events we **read the date off the regulatory calendar** — it is public in advance — and position around it. The four events split into two groups by **how knowable the date is**:

SCHEDULED — date known in advance



UNSCHEDULED

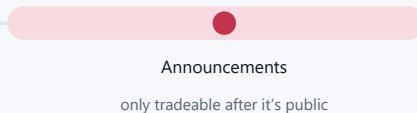


Figure 3. Three events have a date fixed in advance; announcements do not.

1 • QUARTERLY RESULTS

T = broadcast date · scheduled

SEBI LODR Regulation 29 forces the company to intimate the results board meeting **≥ 5 clear days** ahead. That public notice tells you results are coming — it *is* the buy-before window.

2 • BOARD MEETING

T = meeting date · scheduled

Same Regulation 29 intimation: the board-meeting date itself is announced ~5 days ahead. You position into a date the exchange has already published (buybacks, bonuses, fund-raising, results are all decided here).

3 • CORPORATE ACTION

T = ex-date · scheduled

Once a dividend/bonus/split is declared, the **record date & ex-date are broadcast by NSE** (caBroadcastDate) days-to-weeks beforehand — so the date is fixed and public long before you act.

4 • ANNOUNCEMENT

T = disclosure date · unscheduled

Press releases, acquisitions, credit-rating changes, allotments — these arrive **without warning**. You cannot know the date in advance, so there is no genuine “buy-before.” The honest play is the **post-disclosure drift** (the sell-after leg).

The key honesty point for announcements. For the three scheduled events the buy-before leg is real — you position ahead of a known date. For **announcements the buy-before leg is not predictive** (you’d need to know unknowable news early); its win rate should be read only as post-event drift, and treated with extra caution.

Corporate actions use adjusted prices. Adjusted close removes the ex-date drop, so the corporate-action win rate measures **residual total-return drift** around the ex-date — *not* dividend capture or a "recovery" of the drop.

Common thread. For every event the analysis is identical — only the anchor date T and its knowability differ. The prediction is always the same: *this stock's historical hit-rate around this kind of date tends to repeat.*

09 Data-quality work (this build)

Three fixes materially improved the dataset's accuracy:

- A Merged the full history.** The recent NSE export had truncated the financial-results history (e.g. Reliance to 4 quarters). Merging the export with the original archive **recovered +2,860 unique event-dates** — restoring 71–83 quarters per stock and adding older announcements.
- B Fixed board-meeting duplication.** The merge had doubled board meetings (same meeting described two ways). De-duplicating to one row per date **removed 617 duplicate rows** (ratio $2.22\times \rightarrow 1.07\times$).
- C Backfilled board meetings 4×.** NSE's board-meeting feed only holds ~2 recent years. Pulling historical "Outcome of Board Meeting" records from the announcements archive (back to 2006) lifted the average from **~9 to ~38 meetings per stock** — so BOARD_MEETING win-rates rest on a real sample.

Why win-rates dropped slightly after the merge (e.g. an announcement rule $61\% \rightarrow 57\%$): more events regress a flukey small-sample number toward an honest one. Lower-but-more-trustworthy is the right direction.

10 The dashboard & its tabs

A single Python web app (nifty_dash.py, port 8090) serves everything; a static snapshot (docs/nifty/) mirrors it for GitHub Pages.

TAB	SHOWS	LOGIC BEHIND IT
Fundamentals	P&L, balance sheet, ratios, quarterly	Reads the local Screener CSVs; offline Excel export.
 Behaviour	The 64-cell win/return grid + best rule + past-event paths	The core engine per stock.
 Rankings	All 50 stocks ranked by win rate, per event type	Runs the engine across all 50; cached.
 Screener	Filter by ROCE / growth / debt / P/E	Latest annual fundamentals per stock.
 Compare	2–3 stocks side by side	Fundamentals + behaviour together.
Event feeds	Announcements / Board Meetings / Corporate Actions / Financial Results	The raw per-stock CSVs, newest first.

11 Performance & caching

Because the feeds grew large, computing the rankings live took ~40s (and stalled the browser under load). The fix: rankings & screener now **persist to a disk cache keyed by a data fingerprint**, so they load instantly and auto-refresh only when the underlying data changes.

ENDPOINT	BEFORE	AFTER
Rankings	~190 s (hung)	0.09 s
Screener	~2.5 s	0.003 s
Behaviour / feeds	—	< 0.7 s

The cache also warms in a background thread at startup, so the very first click is instant too.

12 Scripts reference

SCRIPT	PURPOSE
event_behaviour.py	The engine — grid of buy-before/sell-after returns & win rates per event.
stock_server.py	The dashboard app: payloads, event feeds, rankings, screener, compare, caching.
nifty_dash.py	Nifty-50 front end on port 8090 (rebrands + warms caches).
download_feeds.py	The Nifty 50 list + NSE feed downloader.
import_new_data.py	Splits the NSE export into per-stock feed CSVs.
merge_event_history.py	Unions the export with the original archive (recovers full history).
enrich_board_meetings.py	Backfills board meetings from the announcements archive.
build_static_nifty.py	Bakes the dashboard into a static GitHub Pages site.

13 Honest limitations & findings

In-sample bias. The best (N,M) is chosen from 64 tries on the same history it is scored on, so win rates flatter. They describe the past; they are not forward probabilities.

Small samples. A 100% win rate on 1–3 events is far weaker than 60% on 30. Always read the win rate next to the event count.

Event overlap. Board meetings and results overlap on dates where the meeting *was* the results approval — so those two event types share some events.

Corporate actions $\neq$ dividend capture (adjusted-close point, §8).

Overall finding. Across honest, market-adjusted testing, no robust, reliably tradeable edge survives out-of-sample on the Nifty 50. The project's real worth is the **transparent framework** and the **clean, verified, fully-historical Nifty 50 dataset** — a solid base for further, more careful research.